

Name: Kanika Saini

SAP ID: 590034098

Program: Btech CSE B16

HDOC Day-17— Digit-Based Operations Using Functions and Switch-Case

1. Problem Statement

Write a menu-driven C program using `switch-case` to perform the following operations on a positive integer:

1. Check whether it is a **Strong Number**.
2. Check whether it is an **Armstrong Number**.
3. Check whether it is a **Harshad Number**.
4. Check whether it is a **Duck Number**.

Each operation must use a **separate user-defined function**, with:

- `void` return type
 - no arguments
 - simple function call
 - input taken inside the respective function
-

2. Algorithm

Main Program Algorithm

Step 1: Start.

Step 2: Display the menu:

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Step 3: Input the user's choice.

Step 4: Use `switch(choice)`.

Step 5: Call the respective function:

- Choice 1 $\rightarrow$ `strongNumber()`
- Choice 2 $\rightarrow$ `armstrongNumber()`
- Choice 3 $\rightarrow$ `harshadNumber()`
- Choice 4 $\rightarrow$ `duckNumber()`

Step 6: For any other choice, display "Invalid choice".

Step 7: Stop.

3. Algorithm for Strong Number

A **Strong Number** is a number in which the **sum of factorials of its digits equals the original number**.

Example:

145

$$1!+4!+5! = 1+24+120 = 145$$

Therefore, **145 is a Strong Number**.

Algorithm

Step 1: Start.

Step 2: Input positive integer `n`.

Step 3: Store `n` in `original`.

Step 4: Initialize `sum = 0`.

Step 5: Extract a digit:

```
digit = n % 10
```

Step 6: Calculate factorial of `digit`.

Step 7: Add factorial to `sum`.

```
sum = sum + factorial
```

Step 8: Remove the last digit:

```
n = n / 10
```

Step 9: Repeat until `n = 0`.

Step 10: Compare `sum` with `original`.

Step 11: If equal, display **Strong Number**; otherwise display **Not a Strong Number**.

Step 12: Stop.

4. Algorithm for Armstrong Number

An **Armstrong Number** is a number in which the sum of each digit raised to the **number of digits** is equal to the original number.

Example:

153

It contains 3 digits.

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153 = 1^3 + 5^3 + 3^3$$

Therefore, **153 is an Armstrong Number**.

Algorithm

Step 1: Start.

Step 2: Input positive integer `n`.

Step 3: Store `n` in `original`.

Step 4: Count the total number of digits.

Step 5: Initialize `sum = 0`.

Step 6: Extract each digit using:

```
digit = n % 10
```

Step 7: Calculate:

```
digit ^ number_of_digits
```

Step 8: Add the result to `sum`.

Step 9: Remove the last digit:

```
n = n / 10
```

Step 10: Repeat for all digits.

Step 11: Compare `sum` with `original`.

Step 12: If equal, display **Armstrong Number**; otherwise display **Not an Armstrong Number**.

Step 13: Stop.

5. Algorithm for Harshad Number

A **Harshad Number** is a number that is completely divisible by the **sum of its digits**.

Example:

18

Sum of digits:

$1+8=9$

Now:

$18 \div 9 = 2$

Since there is no remainder, **18 is a Harshad Number**.

Algorithm

Step 1: Start.

Step 2: Input positive integer n .

Step 3: Store n in `original`.

Step 4: Initialize `sum = 0`.

Step 5: Extract the last digit:

```
digit = n % 10
```

Step 6: Add it to `sum`.

```
sum = sum + digit
```

Step 7: Remove the last digit.

```
n = n / 10
```

Step 8: Repeat until $n = 0$.

Step 9: Check:

```
original % sum == 0
```

Step 10: If true, display **Harshad Number**; otherwise display **Not a Harshad Number**.

Step 11: Stop.

6. Algorithm for Duck Number

A **Duck Number** is a positive number that contains at least one 0 in it, but the zero should **not** be a leading zero.

Examples:

1025 → Duck Number ✓

1203 → Duck Number ✓

1234 → Not a Duck Number ✗

Algorithm

Step 1: Start.

Step 2: Input positive integer n .

Step 3: Initialize `flag = 0`.

Step 4: Extract a digit:

```
digit = n % 10
```

Step 5: Check whether:

```
digit == 0
```

Step 6: If yes, set:

```
flag = 1
```

Step 7: Remove the last digit:

```
n = n / 10
```

Step 8: Repeat until $n = 0$.

Step 9: If `flag == 1`, display **Duck Number**.

Step 10: Otherwise, display **Not a Duck Number**.

Step 11: Stop.

7. C Program

```
#include <stdio.h>

void strongNumber();
void armstrongNumber();
void harshadNumber();
void duckNumber();

int main()
{
    int choice;

    printf("----- DIGIT OPERATIONS MENU -----\\n");
```

```

printf("1. Check Strong Number\n");
printf("2. Check Armstrong Number\n");
printf("3. Check Harshad Number\n");
printf("4. Check Duck Number\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        strongNumber();
        break;

    case 2:
        armstrongNumber();
        break;

    case 3:
        harshadNumber();
        break;

    case 4:
        duckNumber();
        break;

    default:
        printf("Invalid choice\n");
}

return 0;
}

/* Function to check Strong Number */

void strongNumber()
{
    int n, original, digit;
    int sum = 0, fact, i;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    original = n;

    while(n > 0)
    {
        digit = n % 10;

        fact = 1;

        for(i = 1; i <= digit; i++)
        {
            fact = fact * i;
        }
    }

```

```

        sum = sum + fact;

        n = n / 10;
    }

    if(sum == original)
        printf("%d is a Strong Number.\n", original);
    else
        printf("%d is not a Strong Number.\n", original);
}

/* Function to check Armstrong Number */

void armstrongNumber()
{
    int n, original, temp;
    int digit, count = 0;
    int sum = 0, power, i;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    original = n;
    temp = n;

    /* Count number of digits */

    while(temp > 0)
    {
        count++;
        temp = temp / 10;
    }

    temp = n;

    while(temp > 0)
    {
        digit = temp % 10;

        power = 1;

        for(i = 1; i <= count; i++)
        {
            power = power * digit;
        }

        sum = sum + power;

        temp = temp / 10;
    }

    if(sum == original)
        printf("%d is an Armstrong Number.\n", original);
    else
        printf("%d is not an Armstrong Number.\n", original);
}

```



```

/* Function to check Harshad Number */

void harshadNumber()
{
    int n, original, digit;
    int sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    original = n;

    while(n > 0)
    {
        digit = n % 10;
        sum = sum + digit;
        n = n / 10;
    }

    if(original % sum == 0)
        printf("%d is a Harshad Number.\n", original);
    else
        printf("%d is not a Harshad Number.\n", original);
}

/* Function to check Duck Number */

void duckNumber()
{
    int n, original, digit;
    int flag = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    original = n;

    while(n > 0)
    {
        digit = n % 10;

        if(digit == 0)
        {
            flag = 1;
            break;
        }

        n = n / 10;
    }

    if(flag == 1)
        printf("%d is a Duck Number.\n", original);
    else
        printf("%d is not a Duck Number.\n", original);
}

```

}

8. Test Case Table

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	145	Strong Number	Strong Number	Pass
2	1	40585	Strong Number	Strong Number	Pass
3	1	123	Not Strong Number	Not Strong Number	Pass
4	2	153	Armstrong Number	Armstrong Number	Pass
5	2	9474	Armstrong Number	Armstrong Number	Pass
6	2	123	Not Armstrong Number	Not Armstrong Number	Pass
7	3	18	Harshad Number	Harshad Number	Pass
8	3	21	Harshad Number	Harshad Number	Pass
9	3	19	Not Harshad Number	Not Harshad Number	Pass
10	4	1025	Duck Number	Duck Number	Pass
11	4	1203	Duck Number	Duck Number	Pass
12	4	1234	Not Duck Number	Not Duck Number	Pass

9. Compilation and execution

```
(kali㉿kali)-[~/Desktop]
$ cc day17.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
----- DIGIT OPERATIONS MENU -----
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
----- DIGIT OPERATIONS MENU -----
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 40585
40585 is a Strong Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
----- DIGIT OPERATIONS MENU -----
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 123
123 is not a Strong Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
----- DIGIT OPERATIONS MENU -----
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 9474
9474 is an Armstrong Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 123
123 is not an Armstrong Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 21
21 is a Harshad Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 19
19 is not a Harshad Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1025
1025 is a Duck Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1203
1203 is a Duck Number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
— DIGIT OPERATIONS MENU —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1234
1234 is not a Duck Number.
```

10. Evaluation Against Prepared Test Cases

Option 1 — Strong Number

For input 145:

$$1!+4!+5!1!+4!+5! = 1+24+120=145=1+24+120=145$$

The calculated sum equals the original number.

Expected: Strong Number

Actual: Strong Number

Result: PASS ✓

Option 2 — Armstrong Number

For input 153:

Number of digits = 3

$$1^3+5^3+3^3=1+125+27=153=1+125+27=153$$

The calculated sum equals the original number.

Expected: Armstrong Number

Actual: Armstrong Number

Result: PASS ✓

Option 3 — Harshad Number

For input 18:

Sum of digits:

$$1+8=9$$

Check divisibility:

$$18\%9=0$$

Therefore:

Expected: Harshad Number

Actual: Harshad Number

Result: PASS ✓

Option 4 — Duck Number

For input 1025:

Digits are:

1, 0, 2, 5

The number contains 0.

Therefore:

Expected: Duck Number

Actual: Duck Number

Result: PASS ✓

11. Conclusion

The menu-driven C program was successfully implemented using **switch-case and four separate void user-defined functions**. Each function reads its own input and performs the required digit-based operation without passing arguments. The program correctly identifies **Strong, Armstrong, Harshad, and Duck numbers**. All prepared test cases produced the expected results and **passed successfully**.